

Explain Backtracking Code

Absolutely. The key to this problem is to stop thinking "I am arranging numbers" and start thinking "I am filling positions one by one."

1. What is the question asking?

You are given n numbers:

```
1, 2, 3, ..., n
```

You have to arrange **all of them exactly once** into positions:

```
position: 1 2 3 ... n
           [ ? ] [ ? ] [ ? ] ... [ ? ]
```

The arrangement is called **beautiful** if, for every position pos :

```
number at pos is divisible by pos
OR
pos is divisible by number
```

In mathematical form:

```
perm[pos] % pos == 0
OR
pos % perm[pos] == 0
```

Example: $n = 2$

Numbers:

```
1, 2
```

Possible arrangements:

```
[1, 2]
[2, 1]
```

Check $[1,2]$:

```
position 1 → number 1
1 % 1 = 0 ✓

position 2 → number 2
2 % 2 = 0 ✓
```

So $[1,2]$ is beautiful.

Check $[2,1]$:

```
position 1 → number 2
2 % 1 = 0 ✓

position 2 → number 1
2 % 1 = 0 ✓
```

So $[2,1]$ is also beautiful.

Answer:

```
2
```

2. What is the real idea behind your code?

Your code uses **backtracking**.

The basic idea is:

```
Fill position 1, then position 2, then position 3, and so on.
```

At each position, try every number from 1 to n .

But a number can be placed only if:

1. It hasn't already been used.
2. It satisfies the beautiful condition for the current position.

So conceptually:

```
Position 1
  ↓
choose a valid number
  ↓
Position 2
  ↓
choose another valid number
  ↓
Position 3
  ↓
...
  ↓
Position n
```

When every position has been filled:

```
pos > n
```

we found **one complete beautiful arrangement**, so:

```
cnt++;
```

3. Understand `pos`

This is extremely important.

```
solve(pos, n, used, cnt)
```

`pos` means:

Which position am I currently trying to fill?

For example:

```
solve(1)
```

means:

I'm currently filling position 1.

Then:

```
solve(pos + 1, ...)
```

means:

Position 1 is done. Now fill position 2.

So:

```
solve(1)
  ↓
solve(2)
  ↓
solve(3)
  ↓
...
```

4. What does `i` mean?

Inside:

```
for(int i = 1; i <= n; i++)
```

`i` represents the number we're trying to put at the current position.

Suppose:

```
pos = 2
```

Then we are filling:

```
position 2
```

And we try:

```
i = 1
i = 2
i = 3
...
i = n
```

So:

```
if(!used[i] && (pos % i == 0 || i % pos == 0))
```

means:

```
Can I put number i at position pos?
```

5. The most important condition

```
(pos % i == 0 || i % pos == 0)
```

This directly comes from the problem.

Suppose:

```
pos = 3
i = 6
```

Check:

```
3 % 6 = 3
6 % 3 = 0
```

Therefore:

```
6 % 3 == 0
```

So 6 can go into position 3.

Another example:

```
pos = 4
i = 2
```

Check:

```
4 % 2 == 0
```

So 2 can go into position 4.

Therefore:

```
position 4 → number 2
```

is valid.

6. Why do we need `used[]` ?

This is the second most important concept.

Remember:

```
Every number from 1 to n must be used exactly once.
```

Suppose:

```
n = 3
```

We have:

```
1 2 3
```

Imagine we choose:

```
position 1 → number 2
```

Now 2 has already been used.

We cannot later do:

```
position 2 → number 2
```

because then we'd have:

```
[2, 2, ?]
```

which is NOT a permutation.

That's why:

```
vector<int> used(n + 1, 0);
```

creates:

```
index: 0 1 2 3
used:  0 0 0 0
```

We don't care about index 0.

Initially:

```
used[1] = 0
used[2] = 0
used[3] = 0
```

means:

```
1 → unused
2 → unused
3 → unused
```

7. What happens when we choose a number?

Suppose:

```
pos = 1
i = 2
```

and 2 is valid.

We do:

```
used[i] = 1;
```

So:

```
used[2] = 1
```

Now:

```
used:

number: 1 2 3
        ↓ ↓ ↓
used:   0 1 0
```

Meaning:

```
1 → available
2 → USED
3 → available
```

Then:

```
solve(pos + 1, n, used, cnt);
```

We move to the next position.

8. Why do we set it back to 0?

This is the **backtracking** part.

After:

```
solve(...)
```

we do:

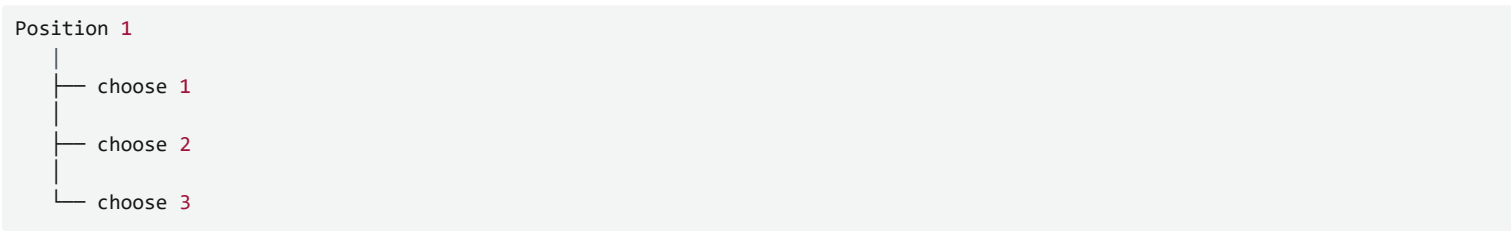
```
used[i] = 0;
```

Why?

Because we are saying:

"I finished exploring all arrangements that start with this choice. Now remove this choice and try another number."

For example:



After completely exploring the 1 branch, we must make 1 available again before trying 2.

So:

```
choose
↓
mark used
↓
explore
↓
unmark
↓
try next
```

That's the essence of backtracking.

9. Now let's dry run n = 3

We have:

```
Numbers = [1, 2, 3]

Positions:

1
2
3
```

Initially:

```
used = [0,0,0,0]
cnt = 0
```

We call:

```
solve(1, 3, used, cnt)
```

10. Position 1

We're at:

```
pos = 1
```

Try every number.

Try `i = 1`

Check:

```
used[1] == 0 ✓
```

```
1 % 1 == 0 ✓
```

So choose:

```
position 1 → 1
```

Mark:

```
used[1] = 1;
```

Current arrangement:

```
[1, _, _]
```

Then:

```
solve(2,...)
```

11. Position 2

Now:

```
pos = 2
```

Current:

```
[1, _, _]
```

Try numbers.

`i = 1`

```
used[1] = 1
```

So:

```
!used[1]
```

is false.

Cannot use 1.

`i = 2`

```
used[2] = 0 ✓
```

Check:

```
2 % 2 == 0 ✓
```

Choose:

```
position 2 → 2
```

Current:

```
[1, 2, _]
```

Mark:

```
used[2] = 1
```

Call:

```
solve(3,...)
```

12. Position 3

Current:

```
[1,2,_]
```

Try numbers.

```
i = 1
```

Already used.

```
i = 2
```

Already used.

```
i = 3
```

Unused:

```
used[3] = 0
```

Check:

```
3 % 3 == 0 ✓
```

Choose:

```
[1,2,3]
```

Then:

```
solve(4,...)
```

13. pos = 4

We have:

```
if(pos > n)
```

Here:

```
4 > 3
```

TRUE.

Therefore:

```
cnt++;
```

So:

```
cnt = 1
```

We found:

```
[1,2,3]
```

as a beautiful arrangement.

14. Now the BACKTRACK happens

We return from:

```
solve(3)
```

and execute:

```
used[3] = 0;
```

So 3 becomes available again.

Then the loop at position 3 continues.

There are no more valid numbers.

So we return to position 2.

Now:

```
used[2] = 0;
```

So 2 becomes available.

15. Position 2 tries another number

At:

```
pos = 2
```

we already have:

```
[1,_,_]
```

Now:

```
i = 3
```

Check:

```
used[3] = 0
```

Good.

Beautiful condition:

```
2 % 3 != 0  
3 % 2 != 0
```

Therefore:

```
3 cannot be placed at position 2.
```

So the `[1,...]` branch is finished.

16. Back to position 1

We now execute:

```
used[1] = 0;
```

Current:

```
used = [0,0,0,0]
```

and try the next number.

17. Position 1 → choose 2

At:

```
pos = 1
```

Try:

```
i = 2
```

Condition:

```
1 % 2 == 0 ✖  
2 % 1 == 0 ✔
```

So valid.

Current arrangement:

```
[2,_,_]
```

Mark:

```
used[2] = 1
```

Go to:

```
pos = 2
```

18. Position 2 with [2,_,_]

Try 1:

```
2 % 1 == 0
```

valid.

So:

```
[2,1,_]
```

Mark:

```
used[1] = 1
```

Go to position 3.

At position 3, only 3 remains.

```
3 % 3 == 0
```

So:

```
[2,1,3]
```

is valid.

Then:

```
pos = 4
```

Therefore:

```
cnt = 2
```

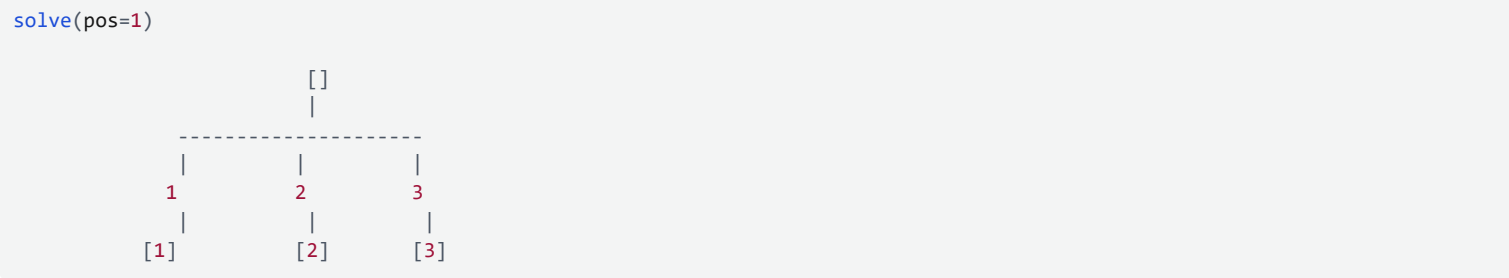
19. Full recursion tree

Now let's visualize the entire recursion tree for n = 3.

I'll write:

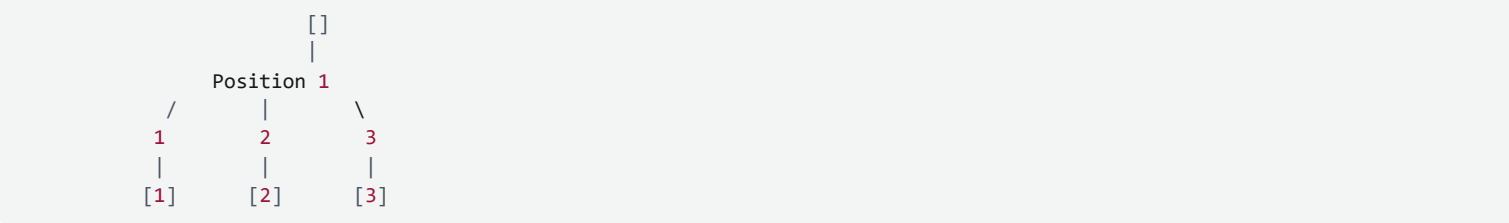
```
(pos -> number chosen)
```

Start

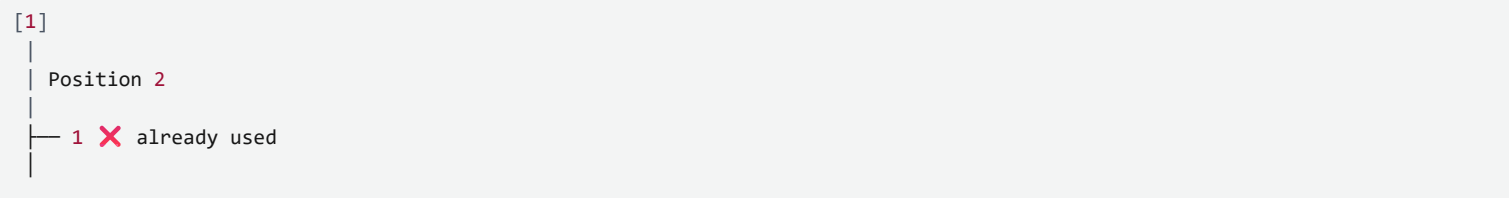


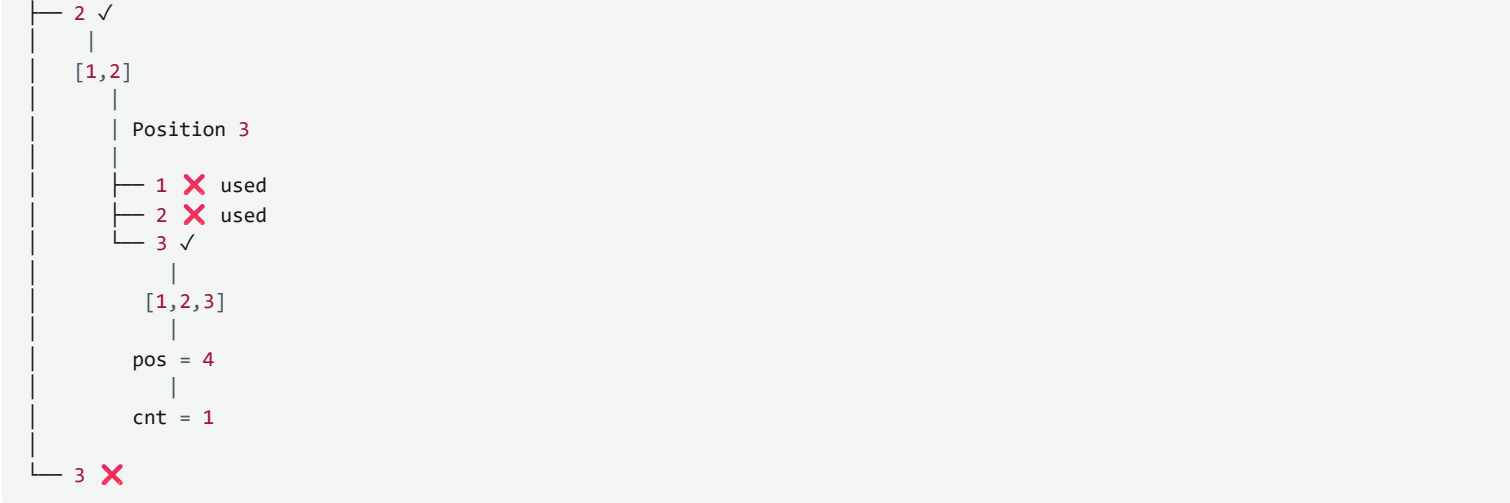
But we must remove invalid branches.

Let's construct it properly.



Branch 1: position 1 → 1

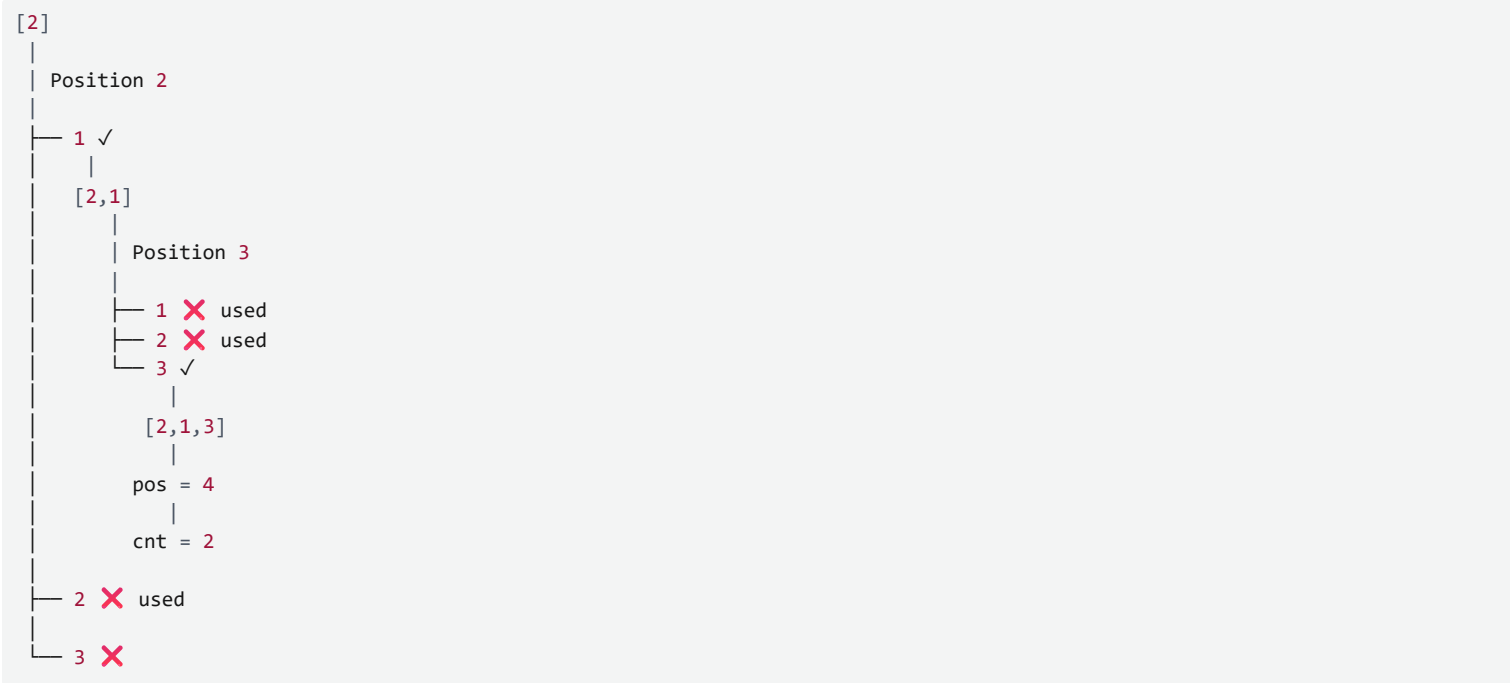




Why is 3 invalid at position 2?

```
2 % 3 != 0
3 % 2 != 0
```

Branch 2: position 1 → 2

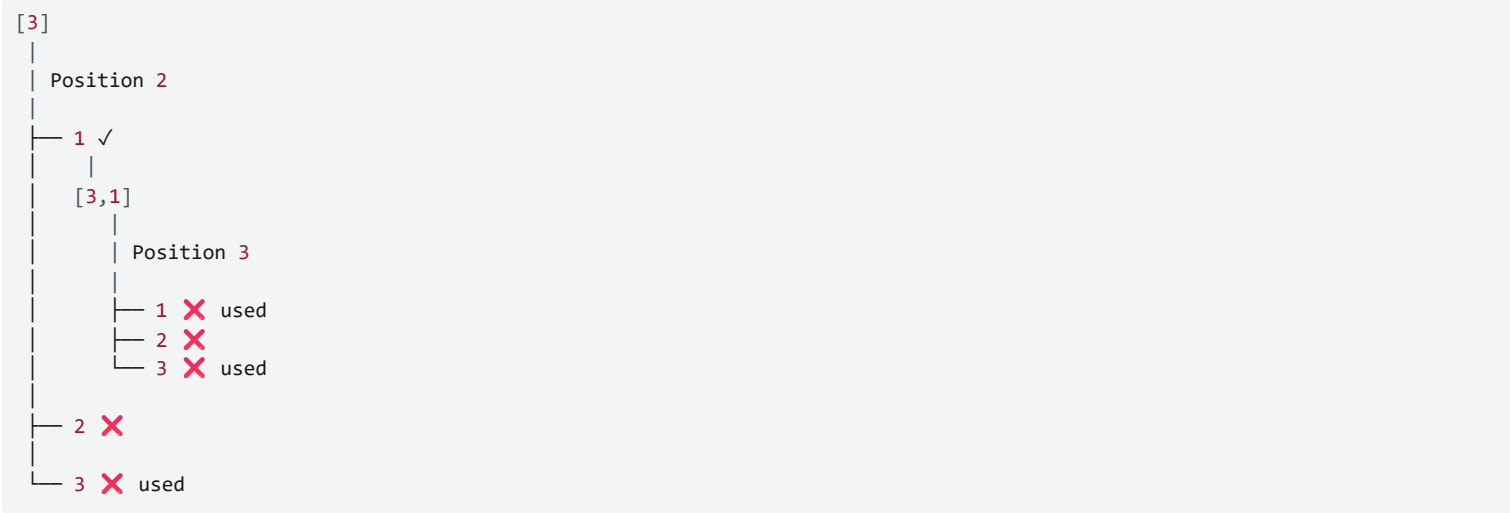


Why is 3 invalid at position 2?

Same reason:

```
2 % 3 != 0
3 % 2 != 0
```

Branch 3: position 1 → 3



Why can't 2 go to position 2?

Actually, 2 can go to position 2, but then:

```
[3,2,_]
```

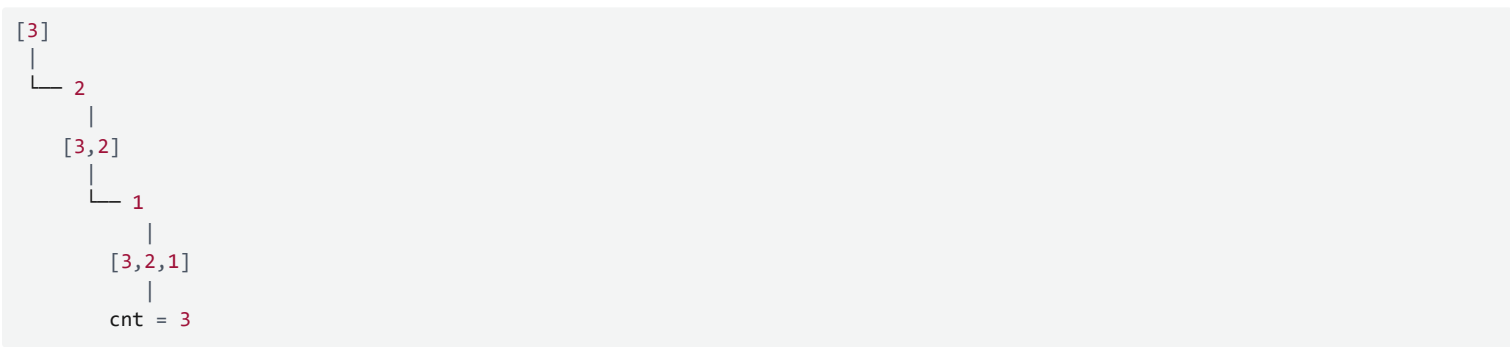
At position 3, only 1 remains, and:

```
3 % 1 == 0
```

So this is ALSO valid:

```
[3,2,1]
```

Therefore the branch is:



So the complete valid arrangements for n = 3 are:

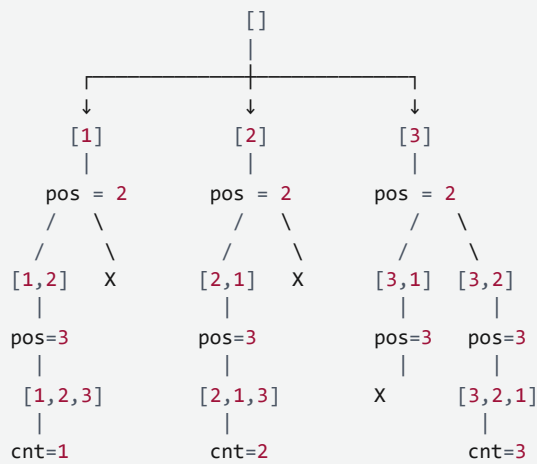
```
[1,2,3]
[2,1,3]
[3,2,1]
```

Therefore:

```
answer = 3
```

20. Complete recursion tree

Putting everything together:



Here:

X = dead branch

because no valid unused number can complete it.

21. Notice something very important

The code does **NOT** explicitly store the arrangement.

There is no:

```
vector<int> ans;
```

Instead, the current arrangement is represented implicitly by:

```
pos
+
used[]
```

For example:

```
pos = 3

used:
1 → used
2 → used
3 → unused
```

tells us that we've already chosen two numbers and we're now filling position 3.

Conceptually, we know the path was something like:

```
1 → 2
```

even though the code doesn't store [1, 2].

Because the problem only asks:

How many arrangements?

we don't need to actually store every arrangement.

We only need:

```
cnt
```

22. The code's execution pattern

This is the most important pattern to understand:

```

for(int i = 1; i <= n; i++) {

    if(valid) {

        used[i] = 1;           // CHOOSE

        solve(pos + 1,...);    // EXPLORE

        used[i] = 0;          // UNCHOOSE
    }
}

```

Think of it as:

```

CHOOSE
↓
mark number used
↓
go deeper
↓
solve next position
↓
RETURN
↓
unmark number
↓
try next number

```

That's **backtracking**.

23. Why does it try 1 to n at every position?

Because we don't know beforehand which number should go there.

For example, at position 1:

```

1 can work
2 can work
3 can work
...
n can work

```

So we explore each possibility.

At position 2, we again check:

```

1
2
3
...
n

```

but `used[]` eliminates numbers already chosen.

So the search is basically:

```

Position 1
├── choose 1
│   ├── Position 2
│   │   ├── choose 2
│   │   │   └── Position 3
│   │   └── choose 3
│   └── choose 2
│       └── Position 2
│           └── choose 3
│               └── Position 2
└── choose 2
    └── Position 2
        └── choose 3
            └── Position 2

```

And invalid choices are simply skipped.

24. One subtle issue in your code

Your base case is:

```
if(pos > n){
    cnt++;
}
```

It works because once `pos > n`, every previous position has already been filled.

But I would normally write:

```
if (pos == n + 1) {
    cnt++;
    return;
}
```

The `return` makes the intention explicit.

Your current version technically continues to the `for` loop after `cnt++`, but since:

```
pos > n
```

and the loop is:

```
for(int i = 1; i <= n; i++)
```

the loop won't find anything valid anyway because the divisibility condition involves `pos`, and the function eventually ends. Still, `return` is cleaner.

So:

```
if (pos > n) {
    cnt++;
    return;
}
```

is better.

25. The entire logic in one picture

Keep this mental model:



And the single most important idea is:

`pos` tells us WHERE we are placing, `i` tells us WHAT number we are trying to place, and `used[i]` tells us whether that number has already been placed.

So for every recursive call:

```
solve(pos, n, used, cnt)
```

you should mentally read it as:

"I have already filled positions `1` through `pos-1`. Now I need to choose a valid unused number for position `pos`."

Once that sentence becomes natural, this entire backtracking solution becomes much easier to understand.